

1. Purpose and Design Principles

This Technical Design Document translates PRD v1.1 into an implementation architecture for the AI Multi-Agent Candidate Evaluation & Hiring System.

The design prioritizes five properties:

- independent agent evaluation before debate;
- evidence-backed findings with source traceability;
- a genuine, bounded debate with response relationships and opinion revision;
- a separate final reasoning stage that does not derive the recommendation through score averaging;
- an interactive professional recruitment interface rather than a generic AI dashboard.

The architecture should remain simple enough for a 3-person student team to understand, implement, test, and demonstrate.

2. Recommended Technology Stack

Frontend: Next.js / React with TypeScript.

Backend: Python + FastAPI.

Database: PostgreSQL.

LLM provider: Gemini or OpenAI API with structured JSON/schema output. The provider should be isolated behind an LLM service interface so it can be changed without rewriting the agents.

Document parsing: Python PDF/DOCX/TXT parsing libraries.

Storage: local development storage, with object storage if deployment requires it.

Orchestration: lightweight custom Python orchestration rather than a heavyweight agent framework.

Deployment: a managed frontend platform plus managed Python backend/database, selected during implementation.

Do not add LangGraph, CrewAI, AutoGen, vector databases, microservices, queues, or other infrastructure unless a concrete requirement demonstrates that it is necessary.

3. High-Level Architecture

The system is organized into five execution stages.

Stage 1 — Neutral Processing

Job Description + Resume + Transcript

→ Document Parser

→ Candidate Profile Builder

→ Evidence Store

→ Job Requirement Extractor

Stage 2 — Independent Evaluation

Shared immutable Candidate Profile + Evidence + Requirements

→ Technical Agent (isolated LLM call)

→ HR/Culture Agent (isolated LLM call)

→ Hiring Manager Agent (isolated LLM call)

→ Skeptic Agent (isolated LLM call)

No independent agent receives another agent's output.

Stage 3 — Debate

Four independent evaluations

→ Debate Engine

→ challenge / response / revision rounds

Stage 4 — Final Decision

Profile + Requirements + Evidence + Independent Evaluations + Debate + Revisions

→ separate Final Decision Agent

Stage 5 — Presentation

Final Decision + Traceability Graph

→ Interactive Final Report

4. Core Backend Components

1. Document Service

- validates uploads;
- extracts text;
- preserves source metadata.

2. Evidence Service

- creates and stores Evidence IDs;
- resolves source locations;
- validates agent quotes;
- maintains evidence-to-finding relationships.

3. Profile Service

- creates the neutral Candidate Profile;
- stores facts and candidate claims;
- never creates hiring judgments.

4. Requirement Service

- extracts job requirements;
- assigns requirement IDs and priorities;
- maps candidate evidence to requirements.

5. Agent Service

- executes the four independent agents;
- validates structured output;
- enforces input boundaries.

6. Debate Service

- exposes independent outputs only after all independent evaluations finish;
- manages debate rounds;
- records response relationships;
- tracks opinion revisions.

7. Decision Service

- calls the separate Final Decision Agent;
- validates final output;
- creates decision-to-evidence traceability.

8. Report Service

- provides structured data for the interactive report.

9. Audit/Execution Service

- records stage, agent, status, timing, validation results, and allowed/excluded input sources.

5. Independence Boundary — Critical Architecture

The independent evaluation stage is the most important architectural boundary.

Each agent call receives only:

- candidate profile;
- job requirements;
- source-linked evidence;
- that agent's system/prompt instructions;
- that agent's evaluation rubric.

It must NOT receive:

- another agent's score;
- another agent's findings;
- another agent's reasoning;
- another agent's recommendation;
- debate messages;
- final decision;
- any aggregate of other agents.

Implementation rule:

The orchestrator creates an immutable `IndependentEvaluationContext` from approved shared inputs. Each agent receives a separate copy/reference of that context plus its own persona configuration. Agent outputs are written to storage only after the independent call completes and are not injected into any other independent call.

The code should make the following sequence explicit:

```
prepare_shared_context()
```

```
→ create_isolated_agent_context(agent)
```

```
→ run_agent(agent)
```

```
→ validate_output()
```

```
→ persist_output()
```

Only after all four calls succeed or are explicitly marked failed may the Debate Service load all independent evaluations.

The system should expose execution metadata showing which sources were allowed and which agent outputs were excluded. This makes independence demonstrable during a project evaluation.

6. Candidate Profile Data Model

The Candidate Profile is a factual shared representation, not an evaluation.

Suggested structure:

CandidateProfile

- id
- candidate_id
- summary_facts
- education[]
- skills[]
- experience[]
- projects[]
- certifications[]
- claims[]
- evidence_ids[]
- created_at
- updated_at

Profile items should reference Evidence IDs.

A claim such as “built a system serving 1 million users” must remain a candidate claim until source evidence or other validation supports it. The Profile Builder must not transform that claim into “has large-scale production experience.”

7. Evidence Data Model

Evidence is a first-class object.

Evidence

- id
- document_id
- source_type
- page
- section
- paragraph
- timestamp
- speaker
- exact_quote
- structured_fact
- evidence_type
- extraction_confidence
- validation_status

Evidence types:

DIRECT_QUOTE

STRUCTURED_FACT

INFERENCE

UNSUPPORTED

Only DIRECT_QUOTE and STRUCTURED_FACT can directly represent source evidence. INFERENCE is an agent interpretation and must point back to its supporting source evidence.

8. Evidence Validation Flow

When an agent returns evidence:

Agent Finding

- Evidence ID / quote
- Evidence Service
- source lookup
- exact or controlled text validation
- VALID / INVALID

If invalid:

1. do not present it as verified evidence;
2. optionally retry the agent once using a structured correction instruction;
3. if still invalid, mark the finding unsupported or insufficiently evidenced.

The UI must clearly distinguish verified source evidence from agent inference.

No agent may create a quotation that is not present in the source.

9. Job Requirement Model

JobRequirement

- id
- job_id
- requirement_text
- category
- priority
- evidence_needed

Priority:

MUST_HAVE

PREFERRED

OPTIONAL

RequirementEvidenceMapping

- requirement_id
- evidence_id
- status
- rationale

Mapping status:

SUPPORTED

PARTIALLY_SUPPORTED

NOT_SUPPORTED

UNKNOWN

The mapping provides structured role-fit information but does not itself make the hiring decision.

10. Agent Execution Design

Each evaluator is implemented as a separate module with:

- persona configuration;
- objective;
- evaluation rubric;
- prompt builder;
- structured output schema;
- validation function.

Recommended modules:

backend/agents/technical.py

backend/agents/hr_culture.py

backend/agents/hiring_manager.py

backend/agents/skeptic.py

backend/agents/final_decision.py

The agents should share infrastructure but not conclusions.

A common BaseAgent abstraction may handle LLM invocation and validation, while each agent defines its own rubric and prompt.

11. Independent Agent Output Schema

Suggested normalized structure:

AgentEvaluation

- id
- evaluation_id
- agent_type
- assessment
- recommendation
- confidence
- confidence_reason
- findings[]
- strengths[]
- concerns[]
- uncertainties[]
- requirement_mappings[]
- created_at

AgentFinding

- id
- evaluation_id
- title
- statement
- severity
- evidence_ids[]
- reasoning
- confidence

Every significant finding must have evidence_ids or explicitly state insufficient evidence.

12. Agent Responsibilities

Technical Agent:

Assess technical skill, depth, practical experience, projects, technical interview answers, and claimed versus demonstrated expertise.

HR/Culture Agent:

Assess role-relevant communication, teamwork, collaboration, leadership, ownership, adaptability, professionalism, and behavioral evidence. Avoid unsupported personality inference.

Hiring Manager Agent:

Assess candidate-job fit, requirement coverage, practical suitability, strengths, risks, and missing requirements.

Skeptic Agent:

Actively search for contradictions, unsupported claims, exaggeration, vague answers, inconsistencies, and missing evidence. Concerns must distinguish proven concern, plausible concern, insufficient evidence, and no concern.

13. Debate Engine

Debate begins only after all independent evaluations are completed or explicitly marked unavailable.

Inputs:

- candidate profile;
- requirements;
- evidence;
- four independent evaluations.

Round 1 — Challenge:

Identify meaningful disagreements and challenge specific findings.

Round 2 — Response:

Agents directly respond to previous messages.

Round 3 — Revision:

Agents state whether their position or confidence changed and why.

Maximum rounds should be configurable, with 1–3 rounds recommended for the MVP.

A debate message must contain:

- message_id;
- round;

- agent;
- statement;
- responding_to_message_id;
- position;
- evidence_ids;
- confidence;
- timestamp.

A response without substantive content does not count as a valid debate response.

14. Opinion Revision

OpinionRevision

- id
- evaluation_id
- agent_type
- original_position
- revised_position
- original_confidence
- revised_confidence
- reason
- triggering_message_ids[]
- evidence_ids[]
- created_at

The system should support no-change outcomes as well. If an agent does not change its position, it must still be able to explain why the opposing evidence did not alter its conclusion.

The UI should make changes visible as:

Original → Challenge → Response → Revised / Unchanged

15. Final Decision Architecture

The Final Decision Agent is separate from all four evaluators.

Input:

- neutral profile;
- job requirements;
- evidence;
- independent evaluations;
- debate messages;
- opinion revisions;
- requirement mappings;
- unresolved disagreements.

The system must NOT compute:

average(scores)
weighted_average(scores)
majority_vote

as the final recommendation.

The final decision is a reasoning output that weighs:

- evidence quality;
- evidence relevance;
- requirement coverage;
- confidence;
- severity of concerns;
- contradictions;
- agreement/disagreement;
- debate outcomes;
- revisions;
- uncertainty.

The Final Decision Agent must explicitly explain why its recommendation was selected over plausible alternatives.

16. Final Decision Schema

FinalDecision

- id
- evaluation_id
- recommendation
- confidence
- confidence_reason
- decision_rationale
- key_supporting_evidence_ids[]
- key_concern_ids[]
- decision_factors[]
- resolved_disagreements[]
- unresolved_disagreements[]
- alternative_considered
- created_at

Recommendation:

STRONG_HIRE

HIRE

CONSIDER

WEAK_CONSIDER

REJECT

Confidence:

HIGH

MEDIUM

LOW

17. Traceability Graph

The core traceability chain is:

Final Decision

→ Decision Factor

→ Agent Finding

→ Evidence ID

→ Source Document

→ Exact Quote / Source Fact

The backend should make these relationships queryable.

Every important final claim should have a path through this chain. If a claim cannot be supported, the report must label it uncertain or unsupported rather than fabricate a source.

18. API Design

Suggested REST endpoints:

POST /evaluations

GET /evaluations/{id}

POST /evaluations/{id}/documents

GET /evaluations/{id}/profile

GET /evaluations/{id}/requirements

GET /evaluations/{id}/evidence

POST /evaluations/{id}/run-independent

GET /evaluations/{id}/agents

GET /evaluations/{id}/agents/{agent_type}

POST /evaluations/{id}/debate

GET /evaluations/{id}/debate

POST /evaluations/{id}/decision

GET /evaluations/{id}/decision

GET /evaluations/{id}/report

GET /evidence/{evidence_id}

Exact request/response schemas should be defined before implementation.

19. Suggested Database Relationships

Candidate 1—M Document

Candidate 1—1 CandidateProfile

CandidateProfile 1—M ProfileItem

Document 1—M Evidence

Job 1—M JobRequirement

JobRequirement M—M Evidence through RequirementEvidenceMapping

Evaluation 1—M AgentEvaluation

AgentEvaluation 1—M AgentFinding

AgentFinding M—M Evidence

Evaluation 1—1 Debate

Debate 1—M DebateMessage

AgentEvaluation 1—M OpinionRevision

Evaluation 1—1 FinalDecision

Foreign keys and unique constraints should prevent cross-evaluation evidence leakage.

20. Frontend Architecture

Suggested application areas:

/dashboard

/evaluations/new

/evaluations/[id]/profile

/evaluations/[id]/agents

/evaluations/[id]/debate

/evaluations/[id]/decision

/evaluations/[id]/evidence

Shared UI components:

- EvidenceDrawer
- AgentCard
- FindingList
- RequirementCoverage
- DebateMessage
- RevisionBadge
- ReasoningTimeline
- DecisionSummary
- TraceabilityPanel

The UI should consume structured backend data rather than reconstructing reasoning from raw LLM text.

21. UI/UX Design Direction

The product should feel like recruitment intelligence software, not a generic AI chatbot.

Visual direction:

- restrained professional palette;
- clear typography;
- strong information hierarchy;

- evidence markers;
- source references;
- timeline and relationship visualization;
- purposeful motion only.

Avoid:

- excessive gradients;
- robot imagery;
- glowing AI effects;
- generic chatbot bubbles;
- meaningless score dashboards.

The central visual story is:

Evidence → Evaluation → Disagreement → Debate → Revision → Decision

22. Interactive Reasoning Experience

Required interactions:

1. Evidence → Findings: clicking evidence shows all findings using it.
2. Finding → Source: clicking a finding opens the source quote.
3. Agent → Debate: clicking a conclusion shows challenges to it.
4. Debate → Evidence: messages expose their evidence.
5. Revision → Cause: revised positions show triggering messages.
6. Final Decision → Factors: decision factors open supporting findings/evidence.
7. Requirement → Evidence: role requirements show mapped candidate evidence.
8. Agent Filter: users can filter debate by persona.
9. Debate Round navigation.
10. Reasoning Timeline navigation.

The goal is for a user to investigate “why?” at every major decision point.

23. Security and Prompt Injection

Uploaded documents are untrusted data.

Resume or transcript content must never override:

- system instructions;
- agent instructions;
- application policies;
- tool permissions.

Use clear delimiters and prompt instructions that explicitly classify document content as data.

API keys remain server-side.

Uploads must be validated.

Temporary files should be cleaned.

Candidate data must remain isolated between evaluations.

Logs should avoid unnecessary sensitive candidate content.

24. Error and Recovery Design

Document failure:

- mark document processing failed;
- show actionable error.

LLM failure:

- retry in a controlled manner;
- if persistent, mark the agent unavailable.

Invalid structured output:

- schema validation;
- controlled repair/retry;
- otherwise reject.

Invalid evidence:

- validate;
- retry if configured;
- otherwise mark unsupported.

Missing agent:

- debate may proceed only if the product explicitly supports partial evaluation;
- final report must clearly state which agent was unavailable.

Final decision failure:

- no fabricated recommendation;
- show decision unavailable and allow retry.

25. Concurrency and Cost

The four independent agent calls can run concurrently because they are independent.

Recommended MVP execution:

- 1 profile builder;
- 2 requirement extraction;
- 3 four independent calls in parallel;
- 4 evidence validation;
- 5 1–3 debate rounds;
- 6 final decision call.

Do not repeatedly send full documents if structured profile/evidence references are sufficient.

The exact model, token limits, retry policy, and temperature should be configurable.

26. Testing Strategy

Unit tests:

- document parsing;

- evidence validation;
- schema validation;
- requirement mapping;
- isolation context creation;
- traceability queries.

Integration tests:

- complete candidate evaluation;
- independent stage;
- debate stage;
- final decision stage;
- report generation.

Critical security/integrity tests:

1. Technical output must not appear in HR independent prompt.
2. HR output must not appear in Technical independent prompt.
3. Debate must not start before independent stage completion.
4. Fake quote must be rejected.
5. Candidate document prompt injection must not alter system instructions.
6. Final decision must work from evidence and debate rather than a score average.
7. Revision must reference an actual triggering debate message.
8. Final claims must resolve to evidence.

27. Demo / Acceptance Test

Use a deliberately mixed candidate case:

- strong technical evidence;
- impressive claims;
- at least one questionable claim;
- at least one resume/transcript inconsistency;
- behavioral evidence;
- enough ambiguity to produce disagreement.

The demonstration should visibly prove:

1. four independent evaluations occurred;
2. independence was preserved before debate;
3. evidence supports findings;
4. one agent directly challenges another;
5. another agent responds;
6. at least one agent changes or qualifies its position;
7. final reasoning considers the disagreement;
8. recommendation is not simple score averaging;
9. final decision traces to source evidence.

28. Implementation Order

Phase 1 — Project setup

Phase 2 — Database and core models

Phase 3 — Document upload and parsing

Phase 4 — Evidence system

Phase 5 — Neutral Candidate Profile Builder

Phase 6 — Job requirement extraction/mapping

Phase 7 — Technical Agent

Phase 8 — HR/Culture Agent

Phase 9 — Hiring Manager Agent

Phase 10 — Skeptic Agent

Phase 11 — Independent orchestration and isolation tests

Phase 12 — Debate Engine

Phase 13 — Opinion Revision

Phase 14 — Final Decision Agent

Phase 15 — Report and traceability

Phase 16 — Interactive UI

Phase 17 — Security/reliability testing

Phase 18 — Deployment and demo preparation

Do not move to the next major phase until the current phase has a working test or demonstrable output.

29. Proposed Repository Structure

candidate-evaluator/

■ ■ ■ ■ frontend/

■ ■ ■ ■ app/

■ ■ ■ ■ components/

■ ■ ■ ■ lib/

■ ■ ■ ■ types/

■ ■ ■ ■ backend/

■ ■ ■ ■ agents/

■ ■ ■ ■ api/

■ ■ ■ ■ models/

■ ■ ■ ■ schemas/

■ ■ ■ ■ services/

■ ■ ■ ■ prompts/

■ ■ ■ ■ orchestration/

■ ■ ■ ■ validation/

■ ■ ■ ■ utils/

■■■ tests/
■ ■■■ unit/
■ ■■■ integration/
■ ■■■ security/
■■■ docs/
■■■ .env.example
■■■ README.md
■■■ docker-compose.yml (optional)

30. Definition of Technical Completion

The technical architecture is considered implemented when:

- the five-stage pipeline works end-to-end;
- independent agent contexts are isolated and testable;
- evidence can be validated against sources;
- all agent outputs are structured;
- debate responses have explicit relationships;
- opinion revisions are recorded;
- final decision is generated by a separate reasoning step;
- no arithmetic aggregation determines the recommendation;
- traceability works from final decision to source;
- the frontend can interactively explore that trace;
- failures are handled without fabricated output;
- the entire flow can be demonstrated with a mixed-evidence candidate.

31. Key Architectural Rule

The project is not primarily a “four chatbot” application.

Its differentiating architecture is:

Neutral evidence extraction

- independent perspectives
- controlled debate
- explicit revision
- evidence-weighted final reasoning
- traceable decision

Every implementation choice should preserve this chain.